



Software Test Automation

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year I Semester

ACADEMIC YEAR : 2025 - 2026

Test Activities & Management

Testing is a structured process to verify code correctness. It contains multiple sequential activities that must be managed, tracked, and reported systematically.

Core Testing Activities

- 1. Test Planning:** Formulating the test strategy, boundaries, environments, and goals.
- 2. Test Preparation:** Designing specific test cases, mapping parameters, and setting up test data.
- 3. Test Execution:** Executing the test cases on build deployments.
- 4. Result Checking:** Comparing actual execution outputs against expected results to detect mismatches.
- 5. Analysis & Follow-up:** Logging bugs in trackers, debugging code, and re-testing fixes.

Managing the Testing Process

- **Role Assignments:** Defining clear responsibilities for test leads, test engineers, and developers.
- **Traceability Matrix:** Mapping test cases to requirement IDs to ensure all features are fully verified.
- **Environment Management:** Provisioning staging environments that match production configurations.
- **Exit Criteria:** Defining clear thresholds (e.g. zero blocker bugs, 90% test pass rate) to release software.

What is Test Automation?

Test Automation uses specialized software tools to execute tests, compare actual outcomes against expected results, and report findings automatically, replacing manual test tasks.

When to Automate (Best ROI)

- **Regression Testing:** Running past tests repeatedly on new builds to check for regression bugs.
- **Repetitive & High-Volume Tests:** Tests running frequently with multiple data variations (data-driven tests).
- **Performance & Load Tests:** Simulating thousands of simultaneous users accessing database connections.
- **Smoke Tests:** Basic validation checks run immediately on every build commit.

When NOT to Automate (Keep Manual)

- **Ad-Hoc / Exploratory Tests:** Dynamic, non-scripted tests looking for edge cases based on tester intuition.
- **Unstable / Evolving Features:** Features undergoing frequent design edits. Constant test script adjustments waste time.
- **One-Off Tests:** Tests run only once or twice. Script development cost exceeds manual execution time.
- **UX & Visual Design Checks:** Assessing visual aesthetics, layout alignment, and user friendliness.

Automation Architecture & Challenges

How automated test frameworks are structured, and their common implementation challenges:

Modular Test Framework Architecture

- **Test Runner:** Orchestrates test runs, maps files, and handles execution sequences.
- **Assertion Library:** Compares actual variables against expected states, reporting passes or failures (e.g. `expect(val).toBe(true)`).
- **Page Object Model (POM):** Design pattern representing web pages as classes. Isolates UI change impacts.
- **Test Data Mocking:** Simulates databases or external APIs using mock services, keeping tests isolated.

Common Automation Challenges

- **Brittle Test Scripts:** Minor UI changes (like changing a button ID) break automated tests, causing high maintenance overhead.
- **Test Data Staleness:** Managing database states so tests run with clean, predictable data consistently.
- **High Initial Setup Cost:** Initial framework setup and script development take substantial developer hours.
- **False Positives / Flakiness:** Tests failing due to network latency, timing issues, or environment glitches rather than actual bugs.

Applications of Test Automation

Where test automation is implemented in professional software delivery pipelines:

CI/CD Integration

Integrates automated test suites into code pipelines. Every pull request automatically runs unit and regression tests before it can be merged.

Automated UI Testing

Uses tools like Selenium or Playwright to simulate user clicks, form submissions, and check page flows across multiple web browsers.

Load & Stress Testing

Uses tools like JMeter or k6 to simulate thousands of concurrent requests, measuring server latency, memory usage, and throughput limits.

API Verification Testing

Automates REST/SOAP endpoint verification. Sends JSON payloads, validates status codes, and checks database state changes.



THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management